IAEmPS2

Um mestre invisível por inteligência artificial para jogos de PlayStation 2 em emulador. Ele vê a tela, lê o estado da partida e intervém pelas alavancas do próprio emulador, dentro de limites declarados, para a experiência ficar mais viva: alivia no perigo real, aperta quando o jogador domina, e muitas vezes não faz nada. Referência de visão, escopo, decisões e roteiro; Casos de Uso e Blueprint Técnico derivam dele.

Nome provisório. "IAEmPS2" é o nome da pasta do protótipo e segue como nome de trabalho; trocar é decisão do cliente. **Revisão 1, 26 de outubro de 2026:** a pedido do cliente, a fase 1 passa a ser tela mais alavancas do emulador, com perfil de memória como camada opcional; Final Fantasy XII sai do escopo; versão própria mínima do emulador decidida; lacunas LC-02 e LC-03 fechadas.

10	3	10	14
módulos funcionais	fases de entrega	regras de negócio	decisões, 8 fechadas e 6 propostas

MÓDULO A

Ponte com o emulador

fase 1

O canal oficial do emulador, estendido com o mínimo que falta: pausar e velocidade. Identificar o jogo, salvar e recarregar estado, ler memória.

MÓDULO B

Tela e situação

fase 1

Capturar a janela do jogo e transformar em situação: onde o jogador está, o que enfrenta, se está parado, perdido ou dominando.

MÓDULO C

Mestre

fase 1

Observar situação e estado, decidir uma alavanca e um motivo, respeitar o ritmo. Tom e intensidade configuráveis antes da sessão.

MÓDULO D

Alavancas e limites

fase 1

O catálogo universal: pausa, câmera lenta, turbo, estado salvo e recarga. Faixas, intervalo mínimo, reversão da última ação.

MÓDULO E

Sessão e overlay

fase 1

Abrir a sessão com qualquer jogo, mostrar situação, ação e motivo na tela, pausar, retomar, encerrar com resumo.

MÓDULO F

Registro

fase 1

Toda captura, decisão e intervenção gravadas com estado antes, ação, estado depois e motivo. Reproduzível e exportável.

MÓDULO G

Processos automáticos

fase 1

Medir custo e latência, tratar falha do serviço de linguagem e do emulador, manter a versão própria do emulador alinhada ao original.

MÓDULO H

Perfil de memória

fase 2

Camada opcional por jogo: variáveis com endereço, tipo e faixa, ações numéricas, fontes públicas de mapas, perfis compartilháveis.

MÓDULO I

Entidades e mundo

fase 3

Posição de inimigos e personagens: aproximar, afastar, reunir. Exige mapa de entidades por jogo.

MÓDULO J

Controle virtual

fase 3

O mestre aciona o controle por um gamepad virtual. Outro tipo de intervenção, com regras próprias.

VISÃO

Olho na tela, mão nas alavancas do emulador

O produto é um diretor de jogo por inteligência artificial: ele não joga e não obedece a pedidos do jogador. Ele vê a tela como um espectador, lê o que o emulador sabe da partida e molda o que acontece, como um mestre de mesa que percebe o grupo entediado e aperta o passo, ou percebe o desespero e dá uma trégua. O protótipo mostrou que vida e dinheiro mudando por decisão de um texto não fazem um mestre; o que faz é entender a situação e agir sobre o ritmo do jogo. Por isso as alavancas da primeira versão são as do próprio emulador, que funcionam em qualquer jogo no primeiro dia.

Mestre (serviço de linguagem)

- Recebe a captura da tela e o estado
- Descreve a situação em uma linha
- Decide uma alavanca do catálogo e um motivo
- Pode decidir "nada"
- Não aciona nada; trocável por configuração

Alavancas

- Universais: pausa, câmera lenta, turbo, estado salvo e recarga
- Vêm do emulador, valem para todo jogo
- Cada uma com faixa, duração e intervalo mínimo
- Opcionais por jogo: ações numéricas do perfil de memória (fase 2)

Plataforma

- Captura a janela e fala com o emulador pela ponte
- Valida cada ação contra o catálogo
- Relê a situação antes de agir
- Registra tudo com motivo
- Testável sem serviço de linguagem

A PRIMEIRA VERSÃO É

- Um jogador, qualquer jogo de PlayStation 2 que ele possua, nenhum mapeamento prévio.
- Overlay sobre a janela do emulador; produto em português do Brasil.
- O ciclo completo: abrir a sessão, o mestre vê e decide, age pelo emulador, o jogador vê, pausa ou reverte, encerra com resumo.
- Uma versão própria do emulador que só acrescenta ao canal oficial o que falta: pausar e velocidade.
- Toda intervenção registrada e reproduzível.

A PRIMEIRA VERSÃO NÃO É

- Escrita na memória do jogo: vida, dinheiro, atributos (fase 2, por perfil).
- Mover inimigos ou personagens no mapa (fase 3).
- Apertar botões pelo jogador (fase 3).
- Trainer com conversa: o jogador não pede efeito.
- Jogador por inteligência artificial, outro emulador ou outro console.

Por que começar pela tela e pelo emulador. Captura de tela, pausa, velocidade e estado salvo existem para qualquer jogo, sem que ninguém mapeie nada; a memória de cada jogo é diferente e exige mapa por título, com cobertura pública desigual. E "o tempo desacelera quando você está prestes a morrer" ou "o mestre te devolve a cena de trinta segundos atrás" é um evento que o jogador sente, enquanto um número subindo é só um número. O perfil de memória continua no roteiro, como camada profunda e opcional, para jogos em que vale o trabalho.

ATORES

Quem usa e o que fica de fora

Ator é quem inicia ou recebe alguma coisa da plataforma. Na primeira versão, o jogador é o cliente deste projeto: ele aprova cada documento e cada sessão jogada é um teste de aceitação. O mapeador só aparece na fase 2, e a separação existe desde já porque o produto é para outros jogadores também, e a maioria deles só vai jogar.

Jogador

`pessoa · ator principal nos módulos C, E`

Abre o emulador com o jogo dele, configura tom e intensidade, joga, vê o que o mestre fez, pausa, reverte e encerra.

Mapeador

`pessoa · ator principal no módulo H (fase 2)`

Cria o perfil de memória de um jogo: variáveis, faixas, ações e proibições. Importa de fontes públicas, valida contra o jogo e publica versões.

Plataforma (rotina)

`sistema · ator principal nos módulos A, B, D, F, G`

O que o produto faz sozinho: capturar, falar com o emulador, validar, agir, registrar, medir, detectar falha.

Serviço de linguagem

`sistema externo · participa de B, C, G`

O modelo que lê a tela, descreve a situação e decide alavanca e motivo. Trocável por configuração; nunca guarda estado, nunca aciona o emulador.

Emulador

`sistema externo · participa de A, D, G`

O PCSX2, em versão própria que só estende o canal oficial de comunicação. Roda o jogo e a BIOS do jogador; o produto não distribui nenhum dos dois.

Fontes de mapas

`sistema externo · participa de H (fase 2)`

Bases públicas de endereços de memória por jogo, mantidas por comunidades. Cobertura e termos de uso variam por título.

DOIS TIPOS DE ALAVANCA QUE NÃO SE MISTURAM

Universais e por jogo

Alavancas universais

Vêm do emulador e valem para qualquer jogo sem preparo: pausar, câmera lenta, turbo, salvar estado num slot reservado ao mestre e recarregar. São a fase 1 inteira. Nenhuma depende de saber o que está em cada endereço de memória.

Perfil de memória

Por jogo, opcional, versionado. Variáveis com significado e faixa, ações numéricas e lista do que nunca se toca. Entra na fase 2, só para jogos em que alguém mapeou ou mapeia, e passa pela mesma validação e pelo mesmo registro das universais.

A sessão viva é o terceiro objeto: o que de fato aconteceu numa partida, captura a captura, decisão a decisão, inclusive "nada". É a fonte de verdade para reverter, retomar e verificar, e é o que separa dirigir de trapacear.

REGRAS DE NEGÓCIO

Dez regras que valem em todos os casos de uso

As cinco primeiras são as regras de mesa: o que mantém a diversão quando o mestre intervém. As demais são regras do produto, e quatro delas nascem de falhas que o protótipo teve de verdade. Cada uma vira teste.

RN-01 **Toda intervenção tem motivo e registro**

Situação, estado antes, ação, estado depois e o motivo do mestre ficam gravados na sessão viva. Intervenção sem registro não existe. Obtemos: mestre auditável.

módulos C, D, F

RN-02 **Dificulta, não derrota**

O mestre pode apertar o ritmo, acelerar, tirar a trégua. Não pode causar derrota direta nem recarregar um estado para fazer o jogador perder o que ganhou. Obtemos: desafio sem roubo.

módulos C, D

RN-03 **Progresso e save são intocáveis**

Cartão de memória e os slots de estado do jogador nunca são tocados. O mestre usa slots reservados a ele, salva antes de qualquer recarga, para que a própria recarga seja reversível, e nunca recarrega sem mostrar. Obtemos: conquista com valor.

módulos A, D

RN-04 **O jogador vê e pode desfazer**

Ação e motivo aparecem no overlay no momento em que acontecem. A última ação pode ser revertida e a sessão pausada a qualquer instante. Obtemos: o controle fica com o jogador.

módulos D, E

RN-05 **Não agir é ação válida**

Há intervalo mínimo entre intervenções e "nada" é uma decisão registrada como qualquer outra. Obtemos: ritmo, não ruído.

módulos C, F

RN-06 **O mestre só age pelo catálogo**

O catálogo é a lista de alavancas universais mais, se houver, as ações do perfil de memória; ele nunca vem do texto do serviço de linguagem, e todo valor fica dentro da faixa declarada. O protótipo somava dinheiro sem teto. Obtemos: nenhuma ação cega.

módulos C, D, H

RN-07 **Serviço de linguagem nunca aciona o emulador**

Ele devolve situação, alavanca e motivo; só a plataforma fala com o emulador, validando contra o catálogo. Obtemos: separação testável sem modelo.

módulos A, C, D

RN-08 **Decidir sobre a situação atual**

A plataforma relê tela e estado antes de agir. Se a situação mudou fora do esperado desde a decisão, a ação é descartada e registrada como tal. O protótipo agia sobre um estado lido segundos antes. Obtemos: nenhuma ação sobre situação velha.

módulos B, D, G

RN-09 **O jogo é do jogador**

Imagem do jogo e BIOS ficam na máquina dele; nunca entram no repositório, nos testes compartilhados nem em qualquer envio. Ao serviço de linguagem vão capturas de tela e o estado resumido, e só isso, com o jogador ciente disso ao abrir a sessão. Obtemos: respeito à propriedade e à licença.

módulos A, B, G

RN-10 **Falha é visível e silencia o mestre**

Serviço fora do ar ou resposta inválida aparece no overlay; sem decisão válida, nenhuma ação acontece; o jogo segue sem o mestre e a sessão retoma. O protótipo ficou meses parecendo vivo com o serviço desligado. Obtemos: nunca parece vivo estando morto.

módulos E, G

ORIGEM

O que o protótipo mostrou e por que a direção mudou

Entre fevereiro e março de 2026 o cliente construiu um protótipo sobre Final Fantasy XII no PCSX2: um overlay que lia vida, energia, dinheiro e atributos do personagem direto da memória do emulador e, a cada ciclo, pedia a um serviço de linguagem uma ação de mestre. Vinte scripts, a maioria ferramentas de varredura de memória. Na avaliação do cliente, não funcionou. Este mapa parte dessa conclusão.

FUNCIONOU COMO MECANISMO

- Ler e escrever a memória do jogo de fora do emulador funciona, em tempo real, sem modificar o emulador.
- O ciclo observar, decidir, agir com um serviço de linguagem no meio roda e devolve decisões com motivo.
- Um overlay sobre a janela do jogo é suficiente como interface: estado, última ação, motivo, histórico.

NÃO FUNCIONOU COMO EXPERIÊNCIA NEM COMO ENGENHARIA

- Com só números na entrada, o mestre decidia o que meia dúzia de regras fixas decidiriam; o jogador via vida e dinheiro mudando, não um evento. É o motivo desta revisão.
- A base da memória era procurada por varredura do valor do dinheiro, com doze scripts para um problema que o canal oficial do emulador resolve.
- Posições de entidades caçadas por diferença de movimento: trinta candidatas, duas plausíveis, nenhuma confiável.
- O serviço de linguagem usado foi descontinuado e o código engolia o erro; endereços de uma sessão e a chave do serviço ficaram gravados no código.

QUATRO LIÇÕES QUE VIRARAM REGRA

Falha silenciosa
A exceção do serviço virava a decisão "nada" com o erro escondido no motivo. Agora é RN-10: falha visível, mestre em silêncio, nenhuma ação.

Estado velho
O estado era lido, o serviço demorava, e a escrita usava o valor antigo. Agora é RN-08: reler antes de agir, descartar se mudou.

Sem teto
Dar dinheiro somava sem limite; atributos tinham tetos fixos no código. Agora é RN-06: catálogo com faixa, nunca vindo do texto do serviço.

Ação dentro do serviço
O serviço escolhia entre ações cujas escritas estavam coladas no mesmo código. Agora é RN-07: o serviço devolve intenção, a plataforma executa e valida.

O que se aproveita. O desenho do overlay e o ciclo de decisão. O que se descarta: toda a varredura de memória, os endereços absolutos, o mapeamento de entidades, a chave gravada no código e a tabela de deslocamentos do personagem, que valia só para uma versão do jogo que o cliente não tem mais.

ROTEIRO DE IMPLEMENTAÇÃO

Três documentos, um portão entre cada, depois código

O projeto é conduzido como processo de engenharia, do protótipo ao produto. Nenhuma etapa começa sem a anterior aprovada pelo cliente. O que for aprendido numa etapa volta para o documento anterior como revisão numerada, não como conversa perdida.

1. Mapa do projeto
Este documento: visão, atores, módulos, regras, decisões e lacunas.

→ **Portão**
Aprovação do cliente.

→ **2. Casos de uso**
Uma ficha por caso: objetivo, fluxo, alternativas, exceções, resultado, regras. Entregue módulo a módulo.

→ **Portão**
Aprovação do cliente.

3. Blueprint técnico
Arquitetura, extensão do canal do emulador, protocolo entre mestre e plataforma, cards de implementação com critério de aceite.

→ **Portão**
Aprovação do cliente.

→ **4. Código, card a card**
Branch por card, revisão, integração contínua verde, critério de aceite provado.

→ **Portão por fase**
Sessões jogadas pelo cliente.

COMO CADA CARD É FECHADO

- Critério de aceite escrito antes do código, no blueprint.
- Plataforma testada isolada, sem serviço de linguagem, contra um emulador simulado que responde ao mesmo canal.
- Mestre testado numa bancada de capturas e estados sintéticos em que toda decisão tem de cumprir as regras de negócio.
- Integração contínua verde é o registro de entrega.
- Mudança de requisito entra como caso de uso novo, com código próprio.

COMO A QUALIDADE É MEDIDA

- Registro de defeitos: cada travada ou intervenção estranha em sessão jogada vira item com causa.
- Medir antes de corrigir: decisão, custo e latência têm número antes de qualquer ajuste.
- Revisão pós-fase: o que funcionou, o que falhou, o que muda no processo.
- Lista de lacunas fechada e republicada no fim de cada fase.

O ESQUELETO AMBULANTE

O primeiro código entregue é a fatia mais fina de ponta a ponta: a versão própria do emulador com pausar e velocidade acrescentados ao canal oficial, a captura da janela, o mestre com quatro alavancas e "nada", o overlay e o registro, rodando em qualquer jogo que o cliente tenha. Ele prova a arquitetura antes de qualquer funcionalidade ganhar corpo, e é o primeiro item que o cliente joga. O que o destrava: um jogo e a BIOS na máquina do cliente (lacuna LC-01) e o ambiente de compilação do emulador (lacuna LC-10).

FASES DE ENTREGA

O que entra em cada fase

A fase 1 é o mestre jogável em qualquer jogo, só com tela e alavancas do emulador. A fase 2 abre a camada de memória, por jogo, e o produto para outros jogadores. A fase 3 dá ao mestre as mãos: entidades e controle.

Fase 1 · mestre jogável em qualquer jogo

módulos A a G

Abrir um jogo no emulador, abrir a sessão, jogar com o mestre vendo a tela e agindo pelas alavancas do emulador dentro das regras de negócio, ver, pausar, reverter, encerrar. Termina quando o cliente joga uma sessão inteira com toda intervenção registrada e nenhuma fora do catálogo.

- A Canal oficial estendido: pausar e velocidade
- A Identificar o jogo
- A Estado salvo em slot reservado
- B Captura da janela
- B Situação em uma linha
- C Observar e decidir
- C Tom e intensidade
- D Catálogo universal com faixas
- D Reverter e pausar
- E Overlay
- E Resumo da sessão
- F Registro reproduzível
- G Falha visível
- G Custo e latência

Fase 2 · perfil de memória e outros jogadores

módulo H e instalação

A camada profunda, por jogo: perfil de memória criado à mão, importado de fontes públicas ou com descoberta assistida, validado contra o jogo em execução, com ações numéricas entrando no mesmo catálogo. Perfis compartilháveis e instalação para outros jogadores. Só começa com a fase 1 jogada, revisada e com a validação da lacuna LC-08 feita.

- H Perfil à mão, validado contra o jogo
- H Importar de fontes públicas
- H Descoberta assistida
- H Publicar e importar perfil
- – Instalação para outros jogadores

Fase 3 · entidades e controle

módulos I, J

Mover inimigos e personagens no mapa, para jogos com mapa de entidades, e acionar o controle por um gamepad virtual. São outros tipos de intervenção, com regras próprias, e só entram sobre registro e catálogo já provados. As regras de negócio continuam valendo: tudo visível, tudo registrado, nada de derrota direta.

- I Posição de entidades
- I Aproximar, afastar, reunir
- J Controle virtual
- J Regras próprias de intervenção por controle

Jogo de abertura. Qualquer jogo de PlayStation 2 que o cliente possua legalmente, com a BIOS do console dele; com alavancas universais, a escolha pesa pouco, e o jogo pode mudar entre sessões sem nada a refazer. Para testes automáticos e integração contínua, um emulador simulado que responde ao mesmo canal, mais um banco de capturas de tela gravadas, para que plataforma e mestre rodem sem emulador, sem jogo e sem serviço de linguagem.

DECISÕES

Oito fechadas, seis propostas

As oito abaixo foram fechadas com o cliente em 26 de outubro de 2026, uma delas revisada no mesmo dia. As seis propostas vêm na página seguinte e fecham com a aprovação deste documento. Mudança posterior entra como revisão numerada, com data, e propaga para os casos de uso e para o blueprint.

CÓDIGO	DECISÃO	ESCOLHA	POR QUÊ	ESTADO
DC-01	Usuário final	Outros jogadores também, não só o cliente	Define desde já perfis compartilháveis e a separação entre quem mapeia e quem joga.	fechada 26/10
DC-02	Escopo da fase 1	Tela e alavancas do emulador; perfil de memória opcional por jogo na fase 2; entidades e controle na fase 3	Números não fizeram um mestre no protótipo; tela e alavancas valem para qualquer jogo no primeiro dia. Substitui a escolha anterior, só valores numéricos.	revisada 26/10
DC-03	Visibilidade	Transparente: overlay com ação e motivo, pausa e reversão	O jogador mantém o controle; é o que torna o mestre aceitável em vez de intrusivo.	fechada 26/10
DC-05	Alvo	Qualquer jogo de PlayStation 2 no PCSX2; o PlayStation 2 é o objetivo, não o meio	Escolha do cliente, ciente de que um jogo de computador com modding daria mais alavancas.	fechada 26/10
DC-06	Ponte com o emulador	O canal oficial de comunicação do emulador; varredura de memória por valor nunca mais	Já oferece leitura e escrita, estado salvo, identificação do jogo e status; elimina os doze scripts do protótipo.	fechada 26/10
DC-07	Emulador	Só PCSX2; outros emuladores e consoles fora	Um emulador, um canal, uma versão própria para manter.	fechada 26/10
DC-13	Versão própria do emulador	Mínima: só o servidor do canal oficial ganha os comandos que faltam, pausar e velocidade; todo o resto igual ao original, com proposta de contribuição ao projeto original	O emulador é código aberto; uma extensão pequena acompanha as versões novas com pouco esforço e pode deixar de ser própria se for aceita.	fechada 26/10
DC-14	Fronteira de licença	O emulador modificado é distribuído com o código, na mesma licença do original; o mestre é programa separado, que fala com o emulador por rede local, com licença própria	Decisão de desenho tomada de propósito: o que estende o emulador herda a licença dele; o que fica fora não. A leitura detalhada da licença é tarefa do blueprint.	fechada 26/10

DECISÕES PROPOSTAS

Seis que fecham com a aprovação deste documento

Cada uma pode ser aceita, alterada ou cortada pelo cliente. O que for alterado entra aqui com a data.

CÓDIGO	DECISÃO	ESCOLHA	POR QUÊ	ESTADO
DC-04	Papel da inteligência artificial	Mestre invisível que molda a partida	Não é jogador por inteligência artificial nem trainer com conversa; é o produto do protótipo.	proposta
DC-08	O jogo é do jogador	O produto não distribui jogo nem BIOS; imagem de jogo nunca entra no repositório	Propriedade e licença são do jogador; o produto só vê a tela e fala com o emulador.	proposta
DC-09	O jogador não pede ação	Configura tom e intensidade antes da sessão; durante, só pausa, reverte e encerra	Se o jogador escolhe o efeito, é trainer. Transparência com pausa e reversão é o limite que mantém o mestre.	proposta
DC-10	Idioma	Produto em português do Brasil; rótulos de perfil podem manter o nome original do jogo	Público do cliente; o nome original evita ambiguidade no mapeamento.	proposta
DC-11	Nome, pasta e repositório	"IAEmPS2" (provisório); pasta Projetos-SSD\iaemps2; repositório privado, branch e merge, nunca commit direto na principal	Mesmo esquema de trabalho dos outros projetos do cliente.	proposta
DC-12	Origem do perfil de memória	À mão, importado de fonte pública ou descoberta assistida; qualquer origem passa por validação contra o jogo em execução antes de entrar numa sessão	Perfil errado age no lugar errado; a validação é a mesma independente da origem.	proposta

FICA PARA O BLUEPRINT TÉCNICO

- Linguagem e estrutura do repositório do mestre; o protótipo é em Python e isso é um ponto de partida, não uma decisão.
- Tamanho exato da extensão do canal do emulador, e como propô-la ao projeto original para zerar o custo de manter uma versão própria.
- Qual serviço de linguagem e qual modelo, decididos com medição de custo, latência e qualidade de leitura de tela e de decisão, nunca por preferência.
- Ciclo de captura e decisão: frequência, resolução da captura, o que vai junto como estado.
- Formato da sessão viva e, na fase 2, do perfil de memória.

LACUNAS

O que ainda não está resolvido

Lista republicada no fim de cada fase. Duas foram fechadas nesta revisão, verificando o emulador instalado na máquina do cliente; a primeira só o cliente fecha.

CÓDIGO	LACUNA	ONDE RESOLVE	RISCO SE FICAR ABERTA
LC-01	Jogo e BIOS na máquina do cliente. Ele não tem mais o jogo do protótipo nem outro; o emulador foi instalado nesta revisão, mas sem BIOS do console dele e sem um jogo que ele possua nada roda. Com alavancas universais, qual jogo pesa pouco.	Cliente, antes do esqueleto ambulante	Esqueleto não anda.
LC-02 fechada 26/10	O emulador instalado expõe a memória pela interface oficial ou pelo símbolo exportado. Verificado no executável da versão 2.8.2: os três símbolos de memória são exportados e o servidor do canal oficial está embutido, com a opção nas configurações avançadas.	Verificação feita nesta revisão	Nenhum.
LC-03 fechada 26/10	Como identificar o jogo em execução para casar com o perfil. O canal oficial devolve título, serial, soma de verificação e versão do jogo por comando próprio.	Código-fonte da versão 2.8.2	Nenhum.
LC-04	Cobertura real e termos de uso das fontes públicas de mapas, jogo a jogo.	Casos de uso do módulo H; levantamento na fase 2	A camada de memória vale para poucos jogos.
LC-05	Como medir a qualidade do mestre sem gabarito. Proposta: bancada de capturas e estados sintéticos em que toda decisão tem de cumprir as regras de negócio, mais sessões jogadas; a diversão em si continua julgamento do jogador.	Blueprint, seção de testes do mestre	Sem medida, regressão passa despercebida.
LC-06	Custo e latência por sessão com captura de tela a cada ciclo, e limite aceitável para o cliente. Imagem custa mais que texto.	Medição no esqueleto ambulante	Produto inviável de operar.
LC-07	Levantamento do que já existe: produtos que alteram jogos em tempo real por evento externo, em emulador ou não. Nenhuma pesquisa feita ainda.	Antes da fase 2	Construir o que já existe.
LC-08	Validação com outros jogadores antes da fase 2: medir interesse e qual jogo pedem primeiro.	Fim da fase 1	Produto certo para uma pessoa, errado para o resto.
LC-09	Nome definitivo do produto.	Decisão do cliente, a qualquer momento	Nenhum.
LC-10	Ambiente de compilação do emulador no Windows (C++, Qt, Visual Studio, dependências), tempo de compilação e custo de acompanhar as versões do projeto original. Nada medido ainda.	Primeiro card do blueprint	Versão própria cara demais de manter; volta a simular tecla de atalho.

Como usar este documento no dia a dia. Ele é a origem de tudo: os casos de uso detalham os módulos daqui, o blueprint implementa os casos de uso, e cada card de código aponta para o caso de uso que cumpre. Quando uma necessidade nova surgir, ela entra primeiro aqui, como módulo, regra, decisão ou lacuna com código próprio, e só depois desce. Assim o escopo continua visível e negociável, e nenhuma decisão fica só na conversa.